

Renta, Ocupación y Actividad en las Secciones Censales de Santa Cruz de Tenerife (2021–2023)

Una historia basada en datos

Proyecto Final de Visualización
Máster Universitario en Ciberseguridad e Inteligencia de Datos
Universidad de La Laguna

Autora Laura Lasso García

Curso 2025–2026

Fecha Mayo de 2026

Repositorio github.com/LauraLasso/dagster-proyecto-final

Resultados lauralasso.github.io/dagster-proyecto-final

Índice

1. Introducción	2
1.1. La historia que se narra	2
2. Fuentes de Datos	3
2.1. Estructura del pipeline	3
2.2. Principios DataOps aplicados	6
3. Checks de Calidad	6
3.1. Check representativo: integridad del join geográfico	7
3.2. Checks del código generado por IA	7
4. Gramática de Gráficos	8
4.1. Componentes comunes a todas las visualizaciones	8
5. Principios de Diseño	8
5.1. Principios Gestalt aplicados	8
5.2. Recomendaciones de diseño de calidad	9
6. Inteligencia Artificial Generativa	9
7. Resultados	10
8. Conclusiones	10
A. Anexo A: Prompts de IA Generativa	13
B. Anexo B: Fragmentos de Código Relevantes	15

Introducción

Este proyecto aborda una cuestión de interés socioeconómico local: ¿cómo se distribuye la renta en las secciones censales de Santa Cruz de Tenerife, y qué relación guarda con la ocupación y la actividad laboral de sus residentes? Para responder a esta pregunta se han integrado cuatro fuentes de datos oficiales —ISTAC e INE— correspondientes al período 2021–2023, construyendo un *pipeline* DataOps completo con Dagster y generando visualizaciones mediante la gramática de gráficos implementada en Plotnine, con asistencia de inteligencia artificial generativa (modelo `llama3.1:8b` vía LiteLLM) para la producción de código de visualización.

La historia que se narra

Santa Cruz de Tenerife no es un territorio uniforme: dentro de la misma ciudad conviven secciones censales con rentas medias muy distintas, estructuras laborales diferentes y niveles de actividad económica dispares. La pregunta que articula este proyecto es la siguiente: **¿Cómo se distribuye la renta en las secciones censales de Santa Cruz de Tenerife entre 2021 y 2023, y qué relación guarda con las fuentes de ingreso y la actividad laboral de sus habitantes?**

Para responderla se cruzan cuatro fuentes de datos oficiales a nivel de sección censal. Una *sección censal* equivale a unas pocas manzanas (entre 1.000 y 2.500 habitantes), lo que permite detectar desigualdades que quedarían ocultas si solo se analizara el municipio en su conjunto.

El relato avanza en cuatro actos:

- Acto I. ¿Cómo evoluciona la renta?** Se parte de la tendencia provincial 2021–2023 para establecer el contexto: ¿sube la renta en general, o se estanca? ¿Aumenta la desigualdad entre secciones?
- Acto II. ¿De dónde vienen los ingresos?** No toda la renta procede del trabajo. En algunos barrios el peso de pensiones y prestaciones sociales supera al de los salarios, lo que revela una estructura económica de mayor dependencia y menor actividad laboral.
- Acto III. ¿Quién trabaja y en qué?** La relación con la actividad muestra qué proporción de la población provincial está ocupada, parada o inactiva a lo largo del período. El mapa de sector predominante de 2023 añade la dimensión territorial: permite identificar qué tipo de actividad económica caracteriza cada sección, aunque sin cuantificar la diversidad sectorial dentro de cada una.
- Acto IV. ¿Dónde están las desigualdades?** Los mapas coropléticos georreferenciados por sección censal permiten ver si la vulnerabilidad socioeconómica se concentra geográficamente —y en qué zonas— o si se distribuye de forma dispersa por el municipio.

La hipótesis de partida es que **las secciones con menor renta media presentan una mayor dependencia de pensiones y prestaciones sociales, y una mayor proporción de población inactiva o desempleada**, configurando territorios de vulnerabilidad socioeconómica concentrada geográficamente. Los datos no permiten confirmar

la diversidad sectorial por sección, pero sí identificar qué tipo de actividad predomina en cada zona y cómo evoluciona la estructura de ingresos a lo largo del período 2021–2023.

Fuentes de Datos

Se han utilizado los cuatro conjuntos de datos indicados en el enunciado, todos a nivel de sección censal, cubriendo el período 2021–2023:

Cuadro 1: Fuentes de datos utilizadas

Dataset	Fuente	Años
Renta bruta media y mediana	ISTAC E30325A_000009	2021–2023
Distribución de renta por fuente	ISTAC E30325A_000002	2021–2023
Sector de ocupación	INE Tabla 70125	2021–2023
Relación con la actividad	INE Tabla 66796	2021–2023

Estructura del pipeline

El pipeline está organizado en **siete grupos de assets**, con dependencias explícitas que Dagster resuelve automáticamente.

El flujo general de datos entre grupos es el siguiente:

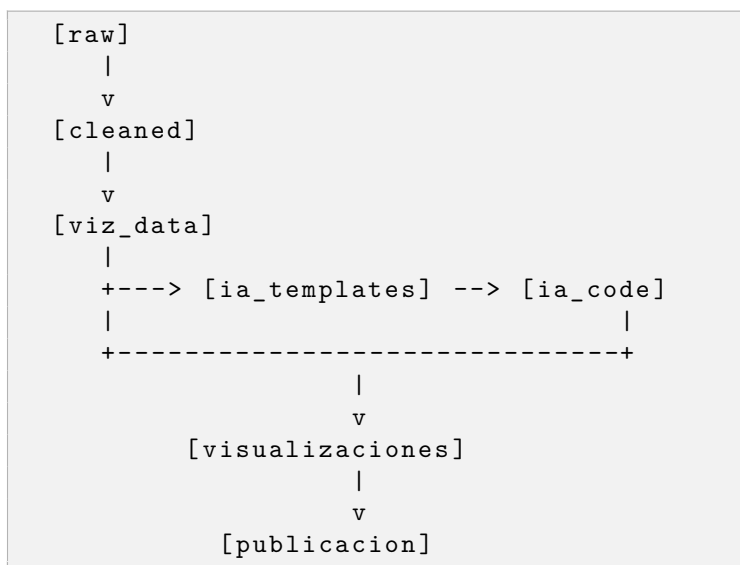


Figura 1: Flujo entre grupos del pipeline

Cada grupo contiene los siguientes assets:

```
[raw]
raw_renta_media          ISTAC E30325A_000009 (CSV)
raw_distribucion_ingresos ISTAC E30325A_000002 (CSV)
raw_actividad            INE 66796 (CSV)
raw_ocupacion            INE 70125 (CSV)
raw_relacion_actividad    proxy actividad-sc-3.csv
raw_geojson_2022/2023/2024 GeoJSON secciones censales
```

Figura 2: Assets del grupo raw: carga de fuentes originales

```
[cleaned]
cleaned_renta_media          filtra medidas, pivota
                             (seccion, anio)
cleaned_distribucion_ingresos detecta col. fuente, normaliza
                             importe
cleaned_actividad            agrupa por
                             seccion/anio/relacion_actividad
cleaned_ocupacion            agrupa por seccion/anio/sector
cleaned_relacion_actividad    extrae CUSEC del geocode,
                             agrupa
```

Figura 3: Assets del grupo cleaned: normalización y limpieza

```
[viz_data]
viz_renta_tendencia          media/mediana/desv provincial por
                             anio
viz_distribucion_ingresos    porcentajes por fuente y anio
viz_actividad_distribucion    porcentajes por categoria y anio
viz_ocupacion_distribucion    porcentajes por sector y anio
viz_relacion_actividad        porcentajes
                             ocupado/parado/inactivo
viz_geo_renta_2021/22/23     GeoDataFrame con join GeoJSON +
                             renta
viz_geo_ocupacion_2023        GeoDataFrame con sector
                             predominante
viz_renta_vs_ocupacion        cruce renta media x total ocupados
```

Figura 4: Assets del grupo viz_data: preparación para graficar

```

[ia_templates] --> [ia_code]
  template_ia_tendencia --> codigo_generado_tendencia
  template_ia_ingresos   --> codigo_generado_ingresos
  template_ia_actividad  --> codigo_generado_actividad
  template_ia_ocupacion  --> codigo_generado_ocupacion

[visualizaciones]
  grafico_tendencia_renta      grafico_distribucion_ingresos
  grafico_actividad            grafico_ocupacion
  mapa_renta_2021/2022/2023    mapa_ocupacion_2023

[publicacion]
  publicar_gh_pages --> git commit + push docs/ (GitHub Pages)

```

Figura 5: Assets de generación IA, visualizaciones y publicación

A continuación se describe en detalle qué hace cada grupo y qué decisiones de implementación se tomaron:

- raw** Carga de las fuentes de datos tal como se reciben, sin ninguna transformación. Todos los CSV se leen mediante la función `leer_csv()`, que detecta automáticamente el separador (, , ; , \t) probando el carácter más frecuente en la primera línea, y la codificación probando en orden `utf-8-sig` (BOM), `utf-8` y `latin-1`. Las columnas se normalizan a minúsculas sin tildes ni espacios. Los tres GeoJSON se cargan con GeoPandas directamente desde disco. El asset `raw_relacion_actividad` reutiliza `actividad-sc-3.csv` como proxy de la tabla INE 66796, hecho que queda registrado en sus metadatos.
- cleaned** Transformaciones que convierten los datos brutos en tablas limpias y consistentes, listas para el análisis posterior:
- **cleaned_renta_media:** filtra las medidas `RENTA_BRUTA_MEDIA_HOGAR` y `RENTA_NETA_UNIDAD_CONSUMO_MEDIANA`, convierte la columna de año a `Int64` y pivota para obtener una fila por (`seccion`, `anio`) con columnas `renta_media` y `renta_mediana`.
 - **cleaned_distribucion_ingresos:** detecta dinámicamente la columna de fuente de renta (puede llamarse `medidas_es`, `tipo` u otras variantes según el CSV), normaliza el importe y elimina filas sin sección o sin valor.
 - **cleaned_actividad y cleaned_relacion_actividad:** extraen el código CUSEC (identificador de sección de 9 dígitos) del geocode completo mediante `split` posicional, normalizan el año y agregan personas sumando ambos sexos con `groupby`.
 - **cleaned_ocupacion:** detecta la columna de sector, extrae el CUSEC del geocode y agrega por (`seccion`, `anio`, `sector`).
- viz_data** Preparación final de los datos para graficar. Se calculan proporciones porcentuales dividiendo cada categoría entre el total del año correspondiente (usando `transform("sum")`). Los assets geoespaciales (`viz_geo_renta_*` y `viz_geo_ocupacion_2023`) realizan un *left join* entre el GeoJSON del año siguiente y los datos limpios, usando como clave el código CUSEC

detectado automáticamente por la función auxiliar `col_cusec()`. El asset `viz_renta_vs_ocupacion` cruza renta media con total de ocupados por sección y año para el gráfico de dispersión.

ia_templates / ia_code Cada `template_ia_*` construye un prompt estructurado en lenguaje de gramática de gráficos (estéticas, geometrías, escalas, tema, principios Gestalt) y llama al LLM local `llama3.1:8b` vía la función `llamar_llm()`. La respuesta se limpia eliminando bloques `"python"`, imports fuera de la función y llamadas con `theme_minimal(figure_size=...)` que rompen Plotnine. El código limpio se devuelve como string en el asset `codigo_generado_*`.

visualizaciones Cada asset intenta ejecutar el código generado por la IA mediante `ejecutar_codigo_ia()`, que hace `exec()` del string en un entorno con Plotnine y Pandas inyectados. Si la ejecución lanza cualquier excepción, se activa automáticamente la función `fallback_*(df, ruta)`, que contiene una implementación Python hardcodeada equivalente. Los mapas coropléticos se generan siempre con Matplotlib + GeoPandas (reproyección a EPSG:3857 para evitar distorsiones de aspecto). Todos los PNGs se guardan a 150dpi y se suben a GitHub con `subir_github()`.

publicacion El asset `publicar_gh_pages` copia todos los PNGs generados al directorio `docs/`, genera un `index.html` estático con las figuras incrustadas, y hace `git add + commit + push` automáticamente. Si no hay cambios respecto al commit anterior, el push se omite para no generar commits vacíos.

Principios DataOps aplicados

- **Reproducibilidad:** todos los pasos son assets Dagster con dependencias declarativas; el pipeline completo se puede relanzar desde cero.
- **Trazabilidad:** cada asset registra metadatos (número de filas, columnas detectadas, años disponibles, tamaño del PNG).
- **Calidad continua:** 26 checks automatizados verifican invariantes en cada etapa (véase Sección 3).
- **Entrega continua:** el asset `publicar_gh_pages` hace `git commit + git push` automáticamente al finalizar el pipeline.
- **Resiliencia:** el patrón IA + *fallback* garantiza que siempre se produce una visualización válida, incluso cuando el LLM genera código erróneo.

Checks de Calidad

Se han implementado **26 checks** con `@asset_check` de Dagster, distribuidos en todos los grupos del pipeline.

Cuadro 2: Distribución de checks por grupo

Grupo	N.º checks	Qué verifican
raw	6	Mínimo de columnas y filas; ausencia de nulos críticos
cleaned	6	Años 2021–2023 presentes; valores positivos; categorías esperadas
viz_data	8	Porcentajes por año suman ≈ 100 %; join geográfico ≥ 50 %
ia_code	4	El código generado contiene <code>def</code> , <code>generarplot</code> , <code>ggplot</code> , <code>return</code> y la geometría esperada
visualizaciones	8	PNG existe en disco y tiene tamaño > 10 KB

Check representativo: integridad del join geográfico

```
@asset_check(asset="viz_geo_renta_2021")
def check_geo_merge_2021(viz_geo_renta_2021):
    pct = float(viz_geo_renta_2021["renta_media"].notna().mean()
                * 100)
    return AssetCheckResult(passed=bool(pct >= 50),
                             metadata={"pct_secciones_con_renta": round(pct, 1)})
```

Este check garantiza que al menos el 50 % de las secciones del GeoJSON han encontrado su correspondiente dato de renta tras el merge, detectando problemas de codificación de CUSEC antes de que lleguen al mapa final.

Checks del código generado por IA

Los checks del grupo `ia_code` verifican la **estructura sintáctica** del código producido por el LLM antes de ejecutarlo:

```
def _check_codigo(codigo: str, geometria: str) ->
    AssetCheckResult:
    tiene_funcion = "def generarplot" in codigo
    tiene_ggplot  = "ggplot"           in codigo
    tiene_return  = "return"           in codigo
    tiene_geom    = geometria          in codigo
    ok = all([tiene_funcion, tiene_ggplot, tiene_return,
              tiene_geom])
    return AssetCheckResult(passed=ok, metadata={...})
```

La razón de verificar la estructura del código es garantizar su ejecutabilidad antes de que llegue al asset de visualización: si el LLM omite la función principal, el parámetro de datos o la geometría requerida, el check falla en `ia_code` y el pipeline activa el fallback Python antes de intentar ejecutar código inválido.

Gramática de Gráficos

Todas las visualizaciones se han construido con Plotnine [5], implementación Python de la gramática de gráficos de Wilkinson [1] popularizada por Wickham [2, 3]. La gramática descompone toda visualización en capas independientes y componibles: datos, estéticas, geometrías, escalas, coordenadas, facetas y temas.

Cuadro 3: Elementos de la gramática de gráficos por visualización

Gráfico	Datos (asset)	Estéticas (aes)	Geometría	Facetas
Tendencia renta	<code>viz_renta_tendencia</code>	<code>x=año, y=media, ymin/ymax=desv</code>	<code>geom_line + geom_ribbon</code>	–
Fuentes de renta	<code>viz_distrib_ingresos</code>	<code>x=fuente, y=%, fill=fuente</code>	<code>geom_bar (dodge)</code>	<code>facet_wrap (año)</code>
Actividad laboral	<code>viz_actividad_distrib</code>	<code>x=categoría, y=%, fill=categoría</code>	<code>geom_bar</code>	<code>facet_wrap (año)</code>
Sector ocupación	<code>viz_ocupacion_distrib</code>	<code>x=sector, y=%, fill=sector</code>	<code>geom_bar</code>	<code>facet_wrap (año)</code>
Renta vs. ocupados	<code>viz_renta_vs_ocupacion</code>	<code>x=ocupados, y=renta, color=año</code>	<code>geom_point + geom_smooth</code>	–
Mapas coropléticos	<code>viz_geo_renta_*</code>	<code>fill=renta_media</code>	<code>geom_sf (matplotlib)</code>	–

Componentes comunes a todas las visualizaciones

Escalas (scale) Se usa `scale_fill_brewer` con paletas cualitativas (`Set2`, `Paired`, `Set3`) para variables nominales, garantizando distinción perceptiva entre categorías. Para la tendencia de renta se emplea `scale_y_continuous` con etiquetas en euros.

Etiquetas Todos los gráficos incluyen título descriptivo, etiquetas de ejes y *caption* con la fuente de datos, siguiendo las recomendaciones de *show the data* de Tufte [4].

Tema `theme_minimal` Se aplica en todos los gráficos para eliminar el ruido visual del fondo gris por defecto, reduciendo el *chartjunk* [4]. El parámetro `figure_size` se especifica siempre dentro de `theme()`, nunca en `theme_minimal()` directamente.

Pequeños múltiples Los gráficos de barras usan `facet_wrap(año)` para comparar los tres años en la misma escala, siguiendo el principio de pequeños múltiples de Tufte [4] que maximiza la comparación sin recodificar.

Principios de Diseño

Principios Gestalt aplicados

Proximidad En los gráficos de barras con `position=dodge`, las barras del mismo año se agrupan visualmente, facilitando la comparación entre fuentes dentro del mismo período.

Similitud El uso consistente de `fill=variable_categorica` con la misma paleta en todos los gráficos hace que categorías iguales se perciban como relacionadas entre sí.

Continuidad En el gráfico de tendencia, la línea guía la vista a lo largo del tiempo (2021 → 2023), apoyada por el *ribbon* de variabilidad que refuerza la lectura como proceso continuo.

Figura-Fondo El *ribbon* semitransparente (`alpha=0.2`) actúa como fondo contextual, destacando la línea de tendencia central como figura principal.

Punto focal En el gráfico de actividad, la categoría *Sueldos y salarios* se colorea en verde, mientras el resto usa tonos suaves y parecidos, creando un punto de atención dirigido.

Recomendaciones de diseño de calidad

- **Relación datos-tinta (*data-ink ratio*):** uso de `theme_minimal` y eliminación de líneas de cuadrícula innecesarias para maximizar la proporción de tinta dedicada a los datos [4].
- **Escala común en facetas:** `facet_wrap` sin `scales="free"` mantiene el eje Y idéntico en todos los paneles, garantizando que las comparaciones entre años sean válidas.
- **Rotación de etiquetas:** `axis_text_x=element_text(angle=45)` en los gráficos con categorías largas evita el solapamiento sin perder legibilidad.
- **Paletas accesibles:** uso de `scale_fill_brewer(type="qual")` con paletas diseñadas para personas con daltonismo.
- **Mapas coropléticos con escala secuencial:** `cmap="YlOrRd"` en los mapas de renta comunica de forma intuitiva la magnitud relativa entre secciones, con amarillo para valores bajos y rojo para valores altos.
- **Caption con fuente:** todos los gráficos incluyen la cita a la fuente original en el margen inferior mediante `labs(caption=...)`.
- **Escala divergente para el gráfico de dispersión `viz_renta_vs_ocupacion`:** el canal `color=año` utiliza tonos progresivos para transmitir la evolución temporal, añadiendo una tercera dimensión sin sobrecargar el gráfico.

Inteligencia Artificial Generativa

Se ha incorporado un modelo LLM local (llama3.1:8b vía Ollama y LiteLLM) para la generación automática de código Plotnine. La arquitectura sigue el patrón IA + *fallback*:

1. El asset `template_ia` construye un prompt estructurado con la descripción del gráfico en términos de gramática de gráficos.
2. El asset `codigo_generado` llama al LLM y limpia el código devuelto (eliminación de imports, postprocesado de `theme_minimal`).

3. El check de `ia_code` valida la estructura del código antes de ejecutarlo.
4. El asset de visualización intenta ejecutar el código generado; si falla, ejecuta el *fallback* Python hardcodeado.

Esta arquitectura garantiza resiliencia: el pipeline siempre produce una visualización válida, incluso cuando el modelo genera código sintácticamente incorrecto.

Resultados

Los gráficos generados están publicados en <https://lauralasso.github.io/dagster-proyecto-final>

Cuadro 4: Visualizaciones generadas por el pipeline

Asset	Tipo	Datos
grafico_tendencia_renta	Línea + ribbon	Renta media 2021–2023
grafico_distribucion_ingresos	Barras + facetas	Fuentes de renta
grafico_actividad	Barras + facetas	Relación con la actividad
grafico_ocupacion	Barras + facetas	Sector de ocupación
mapa_renta_2021/2022/2023	Coroplético	Renta media por sección
mapa_ocupacion_2023	Coroplético	Sector predominante

Conclusiones

El proyecto ha permitido construir una narrativa basada en datos sobre la desigualdad socioeconómica en Santa Cruz de Tenerife a nivel de sección censal. Los principales hallazgos son:

- La renta media provincial muestra una ligera tendencia ascendente entre 2021 y 2023, aunque con alta variabilidad entre secciones.

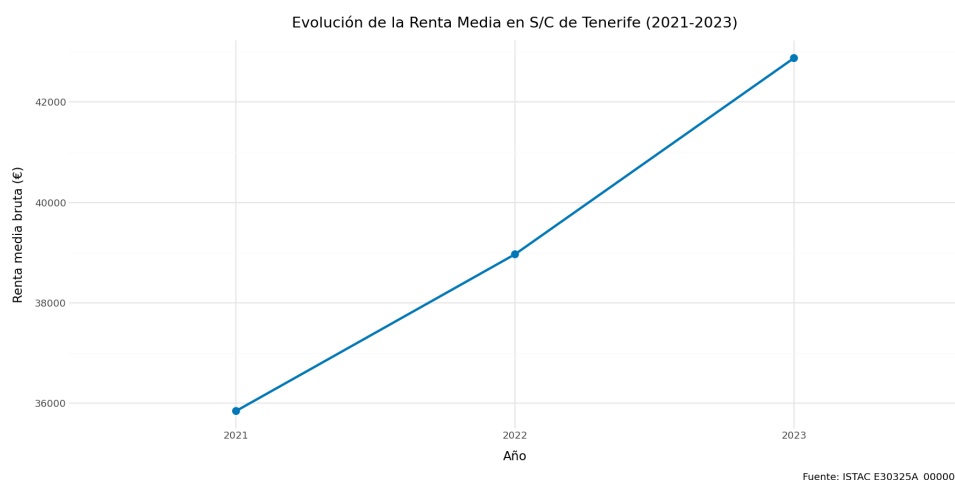


Figura 6: Evolución de la renta media en S/C de Tenerife entre 2021 y 2023, que como se observa tiene una tendencia creciente. Sin embargo, más adelante se observará que hay una gran variabilidad entre secciones censales en las diferentes islas.

- Los salarios y sueldos constituyen la principal fuente de renta en la mayoría de secciones, como se observa en la figura 7, con diferencias significativas en el peso de pensiones y prestaciones entre barrios.

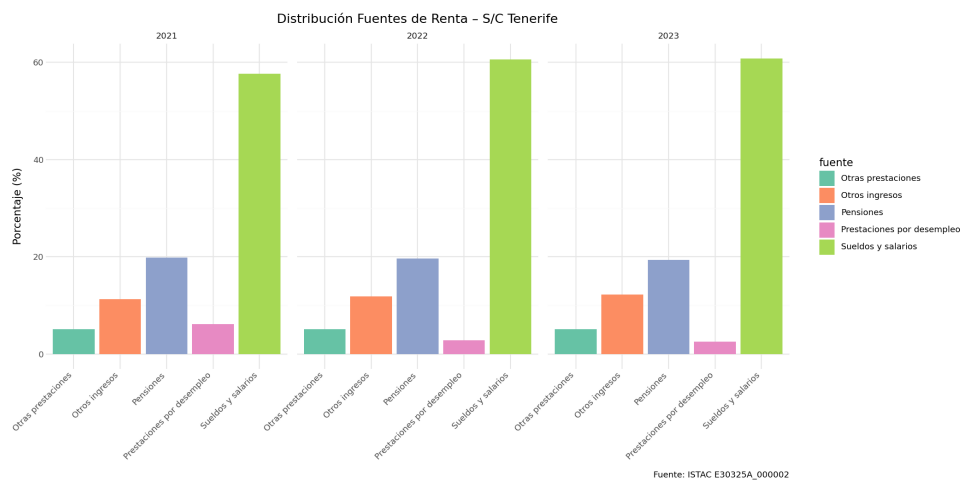


Figura 7: Distribución de fuentes de renta en S/C de Tenerife entre 2021 y 2023, que como se observa, son muy parecidos, con un ligero aumento para sueldos y salarios y una disminución en las pensiones.

- Los datos sugieren una asociación positiva entre la proporción de ocupados y la renta media por sección censal, observable mediante la comparación visual entre los mapas coropléticos de renta y el mapa de sector predominante de 2023.

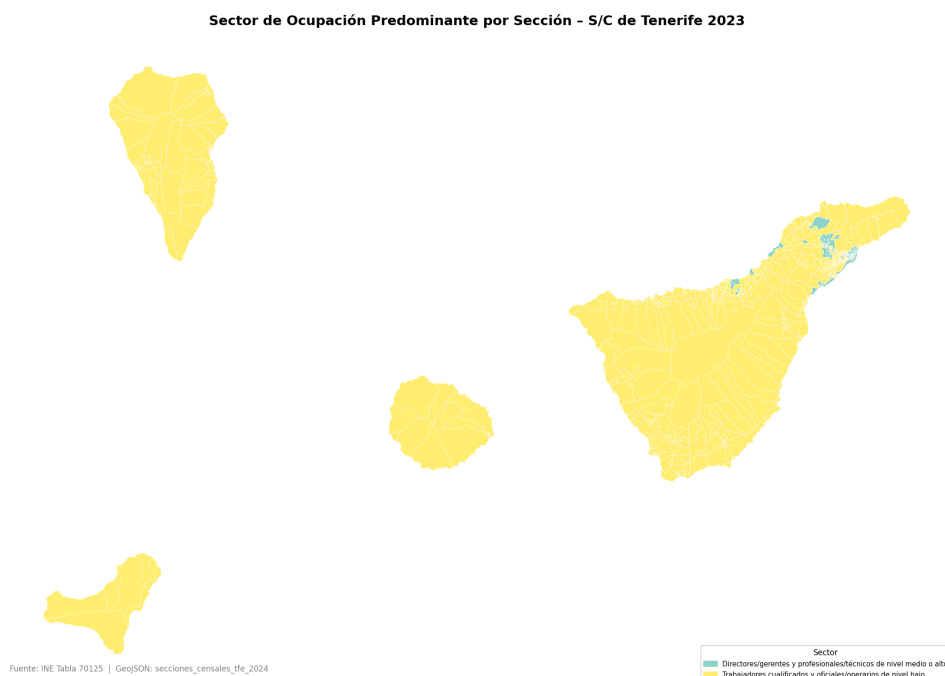


Figura 8: Sector de ocupación predominante por sección en S/C de Tenerife en 2023.

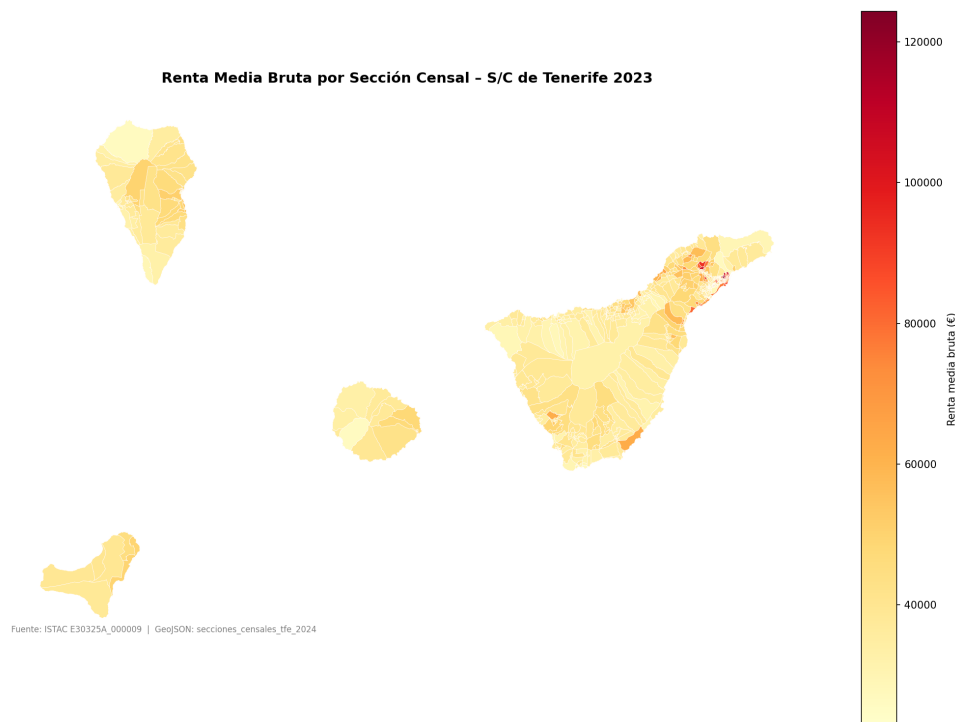


Figura 9: Renta media bruta por sección censal en S/C de Tenerife en 2023.

En respuesta a la pregunta planteada, los datos revelan un panorama claro y consistente a lo largo del período 2021–2023. Como muestra el gráfico de tendencia de renta (Fig. 6), la renta media provincial crece de forma sostenida, pasando de aproximadamente 36.000 € en 2021 a 43.000 € en 2023. El gráfico de distribución de fuentes de renta (Fig. 7) revela que este crecimiento está impulsado principalmente por los sueldos y salarios, que representan entre el 58 % y el 61 % de los ingresos totales y aumentan su peso relativo año a año.

Los mapas coropléticos de renta por sección censal muestran que este crecimiento no se distribuye de forma homogénea, para el año 2023 por ejemplo, en la figura 9: las secciones con renta más alta se concentran de manera persistente en la zona norte y noreste de la provincia. Esta concentración geográfica es coherente con el mapa de sector de ocupación predominante de 2023 (Fig. 8), donde esas mismas secciones son las únicas en las que predominan los directores, gerentes y profesionales técnicos de nivel medio o alto, frente al resto del territorio, donde el sector predominante son los trabajadores cualificados de nivel bajo, tal y como confirma también el gráfico de sector de ocupación provincial.

El gráfico de actividad económica muestra que el sector servicios domina con cerca del 85 % del empleo, sin variación significativa entre años. Por su parte, las pensiones representan en torno al 20 % de los ingresos de forma estable (Fig. 7), lo que indica la presencia de una población envejecida cuya renta no depende del mercado laboral. Las prestaciones por desempleo, en cambio, reducen su peso del 7 % al 3 % entre 2021 y 2023, señal de una mejora en el empleo a nivel provincial. En conjunto, los datos apuntan a que la vulnerabilidad socioeconómica se concentra geográficamente en las secciones alejadas del núcleo urbano norte, donde el empleo cualificado es escaso y el peso de las prestaciones es comparativamente mayor.

Anexo A: Prompts de IA Generativa

A continuación se detallan los prompts enviados al LLM para cada visualización. En todos ellos se emplea terminología de la gramática de gráficos (estéticas, geometrías, escalas, facetas, temas) y se incorporan principios de diseño de Tufte y Gestalt, en línea con los contenidos de la asignatura.

Sistema base (común a todos los prompts)

```
SYSTEM_BASE = """
Eres un experto en gramática de gráficos y Plotnine.
Devuelve EXCLUSIVAMENTE el bloque:
    def generarplot(df):
        from plotnine import ...
        plot = ggplot(df, aes(...)) + ...
        return plot
Sin ningún import fuera de la función, sin texto antes ni
después,
sin bloques ```python```.
NUNCA uses theme_minimal(figure_size=...).
Usa theme_minimal() + theme(figure_size=...) separados.
"""
```

Prompt 1 — Gráfico de tendencia de renta

```
descripcion = """
- Dataset viz_renta_tendencia con columnas: anio,
  media_provincial,
  mediana_provincial, desv_std
- Años disponibles: [2021, 2022, 2023]
- Estéticas: aes(x='anio', y='media_provincial')
- Geometrías:
    geom_ribbon(aes(ymin='media_provincial'-'desv_std',
                    ymax='media_provincial'+'desv_std'),
                fill='#0077b6', alpha=0.2)
    geom_line(color='#0077b6', size=1.2)
    geom_point(color='#0077b6', size=3)
- Escalas: scale_y_continuous con etiquetas en euros.
            scale_x_continuous con breaks=[2021, 2022, 2023].
- Etiquetas: labs(title='Evolución de la Renta Media en SC de
  Tenerife
                    2021-2023', x='Año', y='Renta media bruta',
                    caption='Fuente: ISTAC E30325A_000009')
- Tema: theme_minimal(). figure_size=(12,6) DENTRO de theme().
- Principios Gestalt: Continuidad (línea), Figura-Fondo (ribbon).
- NO usar matplotlib.
"""
```

Justificación: El prompt especifica la geometría `geom_ribbon` para representar la variabilidad como banda de confianza (principio Gestalt de *Figura-Fondo*), la escala continua en el eje Y con unidades monetarias, y el principio de *Continuidad* mediante la línea de tendencia.

Prompt 2 — Distribución de fuentes de renta

```
descripcion = """
- Dataset viz_distribucion_ingresos con columnas: anio, fuente,
  importe, porcentaje
- Fuentes disponibles: [sueldos_salarios, pensiones,
  prestaciones, ...]
- Estéticas: aes(x='fuente', y='porcentaje', fill='fuente'),
  facet por anio.
- Geometría: geom_bar(stat='identity', position='dodge')
- Escalas: scale_fill_brewer(type='qual', palette='Set2')
  scale_y_continuous(limits=(0, 60)) con etiquetas en %
- Etiquetas: labs(title='Distribución de Fuentes de Renta
  Secciones
  SC de Tenerife', caption='Fuente: ISTAC
  E30325A_000002')
- Tema: theme_minimal(). figure_size=(14,7) DENTRO de theme().
  Rotar eje X 45 grados:
  axis_text_x=element_text(angle=45).
- Gestalt: Similitud (color por fuente), Proximidad (barras
  agrupadas).
- Pequeños múltiples Tufte: facet_wrap('anio') misma escala.
- NO usar matplotlib.
"""
```

Justificación: Se aplica el principio de pequeños múltiples de Tufte [4] mediante `facet_wrap`, se usa una paleta cualitativa accesible y se especifica `position=dodge` para aprovechar el principio Gestalt de Proximidad.

Prompt 3 — Relación con la actividad

```
descripcion = """
- Dataset viz_actividad_distribucion con columnas: anio,
  relacion_actividad, personas, porcentaje
- Categorías: [Ocupado, Parado, Inactivo, Otro]
- Estéticas: aes(x='relacion_actividad', y='porcentaje',
  fill='relacion_actividad'), facet por anio.
- Geometría: geom_bar(stat='identity')
- Escalas: scale_fill_manual con paleta accesible:
  Ocupado='#0077b6', Parado='#e63946',
  Inactivo='#457b9d', Otro='#a8dadc'
  scale_y_continuous con etiquetas en %
- Punto Focal Gestalt: Ocupado en azul intenso, resto en tonos
  suaves.
```

```
- Etiquetas: labs(title='Relación con la Actividad - Secciones
    SC de
                Tenerife 2021-2023', caption='Fuente: INE
                Tabla 66796')
- Tema: theme_minimal(). figure_size=(14,7) DENTRO de theme().
- NO usar matplotlib.
"""
```

Justificación: Se utiliza `scale_fill_manual` para asignar colores semánticamente significativos: el azul intenso para *Ocupado* crea un punto focal Gestalt, guiando la atención hacia la variable de mayor interés analítico.

Prompt 4 — Sector de ocupación

```
descripcion = """
- Dataset viz_ocupacion_distribucion con columnas: anio, sector,
  personas, porcentaje
- Sectores disponibles: [Servicios, Comercio, Construcción, ...]
- Estéticas: aes(x='sector', y='porcentaje', fill='sector'),
  facet por anio.
- Geometría: geom_bar(stat='identity')
- Escalas: scale_fill_brewer(type='qual', palette='Paired')
  scale_y_continuous con etiquetas en %
- Etiquetas: labs(title='Sector de Ocupación - Secciones SC de
  Tenerife
                2021-2023', caption='Fuente: INE Tabla 70125')
- Tema: theme_minimal(). figure_size=(14,7) DENTRO de theme().
  Rotar eje X 45 grados.
- NO usar matplotlib.
"""
```

Anexo B: Fragmentos de Código Relevantes

Lectura robusta de CSV con detección automática de separador

```
def leer_csv(path: Path) -> pd.DataFrame:
    """Lee un CSV detectando automáticamente el separador y
    codificación."""
    for enc in ['utf-8-sig', 'utf-8', 'latin-1']:
        try:
            primera =
                path.read_text(encoding=enc).splitlines()[0]
            enc_detectada = enc
            break
        except Exception:
            continue
    candidatos = {' ','': primera.count(',')}, {';':
        primera.count(';')},
```

```
        '\t': primera.count('\t')}\n    sep = max(candidatos, key=candidatos.get)\n    df = pd.read_csv(path, sep=sep, encoding=enc_detectada,\n                    dtype=str, engine='python',\n                    on_bad_lines='skip')\n    df.columns = (df.columns.str.strip().str.lower()\n                  .str.replace(r'[á  ]', 'a', regex=True)\n                  ...)\n\n    return df
```

Patrón IA + fallback para generación de gráficos

```
@asset(group_name="visualizaciones")\ndef grafico_tendencia_renta(context, codigo_generado_tendencia,\n                             viz_renta_tendencia):\n    ruta = str(OUTDIR / "grafico_tendencia_renta.png")\n    try:\n        ejecutar_codigo_ia(codigo_generado_tendencia,\n                           viz_renta_tendencia).save(ruta, ...)\n    except Exception as e:\n        context.log.warning(f"Fallo IA fallback: {e}")\n        fallback_tendencia(viz_renta_tendencia, ruta)\n    subir_github(ruta)\n    return Output(ruta, metadata={"url":\n                                   MetadataValue.url(ruta)})
```

Referencias

- [1] Wilkinson, L. (2005). *The Grammar of Graphics* (2nd ed.). Springer.
- [2] Wickham, H. (2010). A Layered Grammar of Graphics. *Journal of Computational and Graphical Statistics*, 19(1), 3–28.
- [3] Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis* (2nd ed.). Springer.
- [4] Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2nd ed.). Graphics Press.
- [5] Kibirige, H. et al. (2023). *plotnine: A Grammar of Graphics for Python*. <https://plotnine.readthedocs.io>
- [6] Dagster Labs (2024). *Dagster: The Cloud-Native Orchestrator*. <https://dagster.io>
- [7] Ereth, J. (2018). DataOps: Towards a Definition. *LWDM Workshop, EDBT/ICDT*.
- [8] ISTAC (2023). *Distribución de la renta en las secciones censales de Canarias*. E30325A_000009 y E30325A_000002. Gobierno de Canarias. <https://www3.gobiernodecanarias.org/istac>
- [9] INE (2023). *Censo anual de población: ocupación y actividad. Tabla 70125*. Instituto Nacional de Estadística. <https://www.ine.es/jaxiT3/Tabla.htm?t=70125>

- [10] INE (2023). *Población de 16 y más años por relación con la actividad. Tabla 66796*. Instituto Nacional de Estadística. <https://www.ine.es/jaxiT3/Tabla.htm?t=66796>